18

Advanced Next.js Concepts and Optimizations

Now that we've learned about the essential features of Next.js and **React Server Components** (**RSCs**), let's dive a bit deeper into the Next.js framework. In this chapter, we are going to learn how caching works in Next.js and how it can be used to optimize our applications. We are also going to learn how to implement API routes in Next.js. Then, we are going to learn how to optimize a Next.js app for search engines and social media by adding metadata. Finally, we are going to learn how to optimally load images and fonts in Next.js.

In this chapter, we are going to cover the following main topics:

- Defining API routes in Next.js
- Caching in Next.js
- **Search engine optimization** (**SEO**) with Next.js
- Optimized image and font loading in Next.js

Technical requirements

Before we start, please install all the requirements from ***Chapter 1***, *Preparing For Full-Stack Development*, and ***Chapter 2***, *Getting to Know Node.js and MongoDB*.

The versions listed in those chapters are the ones used in this book. While installing a newer version should not be an issue, please note that certain steps might work differently. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in ***Chapter 1*** and *2*.

You can find the code for this chapter on GitHub: https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch18.

The CiA video for this chapter can be found at: https://youtu.be/jzCRoJPGoG0.

Defining API routes in Next.js

In the previous chapter, we used RSCs to access our database via a data layer; no API routes were needed for that! However, sometimes, it still makes sense to expose an external API. As an example, we might want to allow third-party apps to query blog posts. Thankfully, Next.js also has a feature to define API routes, called Route Handlers.

Route Handlers are also defined inside the **`src/app/`** directory but in a **`route.js`** file instead of a **`page.js`** file (a path can only be either a route or a page, so only one of these files should be placed inside a folder). Instead of exporting a page component, we need to export functions that handle various types of requests there. For example, to handle a **`GET`** request, we must define and export the following function:

```
export async function GET() {
```

Next.js supports the following HTTP methods for Route Handlers: **`GET`**, **`POST`**, **`PUT`**, **`PATCH`**, **`DELETE`**, **`HEAD`**, and **`OPTIONS`**. For unsupported methods, Next.js will return a **`405 Method Not Allowed`** response.

Next.js supports the native **`Request`** (https://developer.mozilla.org/en-US/docs/Web/API/Request) and **`Response`** (https://developer.mozilla.org/en-US/docs/Web/API/Response) web APIs but extends them into **`NextRequest`** and **`NextResponse`** APIs, which make handling cookies and headers easier. We used the **`cookies()`** function from Next.js to easily create, get, and delete a cookie for the JWT in the previous chapter. The **`headers()`** function makes it easy to get headers from a request. These functions can be used in the same way in RSCs and Route Handlers.

Creating an API route for listing blog posts

Let's start by defining an API route for listing blog posts:

1. Copy the existing **`ch17`** folder to a new **`ch18`** folder, as follows:

```
$ cp -R ch17 ch18
```

2. Open the **`ch18`** folder in VS Code.
3. To make the API routes easier to distinguish from pages on our app, create a new **`src/app/api/`** folder.
4. Inside the **`src/app/api/`** folder, create a new **`src/app/api/v1/`** folder to make sure our API is versioned for potential changes to the API later.
5. Next, create a **`src/app/api/v1/posts/`** folder for the **`/api/v1/posts`** route.
6. Create a new **`src/app/api/posts/route.js`** file, where we import the **`initDatabase`** function and the **`listAllPosts`** function from the data layer:

```
import { initDatabase } from '@/db/init'
import { listAllPosts } from '@/data/posts'
```

7. Then, define and export a **`GET`** function. This is going to handle HTTP GET requests to the **`/api/v1/posts`** route:

```
export async function GET() {
```

8. Inside it, we must initialize the database and get a list of all posts:

```
  await initDatabase()
  const posts = await listAllPosts()
```

9. Use the **`Response`** web API to generate a JSON response:

```
  return Response.json({ posts })
}
```

10. Make sure Docker and the MongoDB container are running properly!
11. Start the Next.js app, as follows:

```
$ npm run dev
```

12. Now, go to **`http://localhost:3000/api/v1/posts`** to see the posts being returned as JSON, as shown in the following figure:

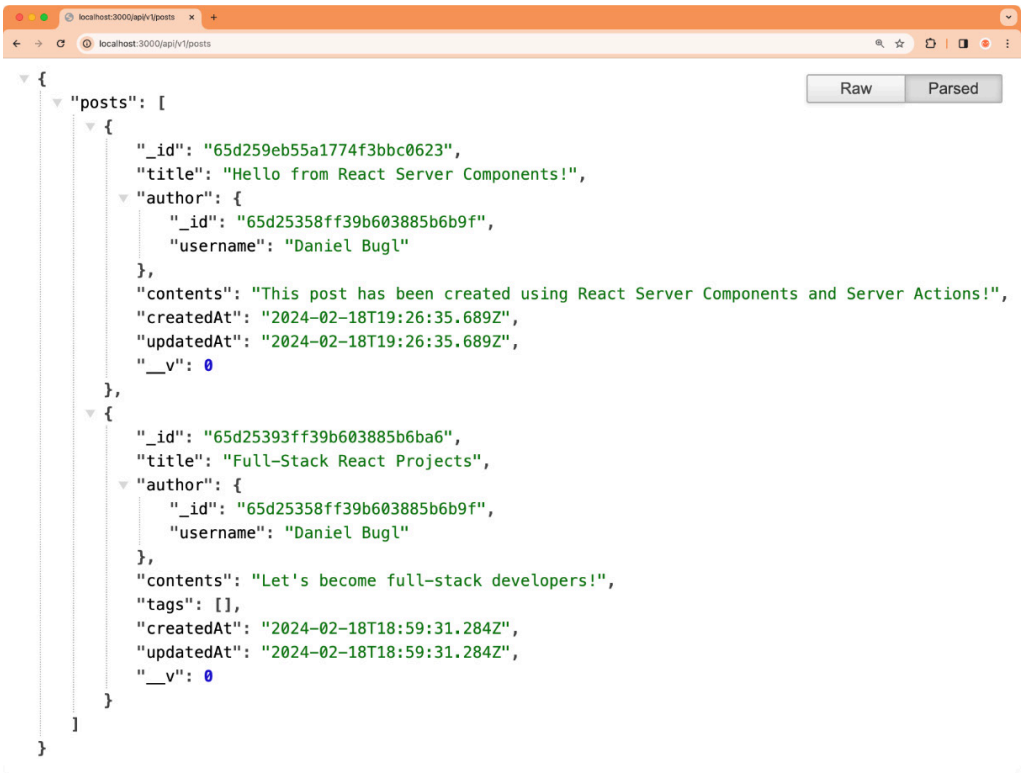

Figure 18.1 – JSON response with posts generated from the Next.js Route Handler

Now, third-party apps can also get the posts via our API! Let's continue by learning more about caching in Next.js.

Caching in Next.js

So far, we have always been using Next.js in dev mode. In dev mode, most of the caching that Next.js does is turned off to make it easy for us to develop our apps with hot reloading and always up-to-date data. However, once we switch to production mode, static rendering and caching are turned on by default. Static rendering means that if a page only contains static components (such as an "About Us" or "Imprint" page, which only contains static content), it will be statically rendered and served as HTML, or as static text/JSON for routes. Additionally, Next.js will try to cache data

and server-side rendered components as much as possible to keep your app performant.

Next.js has four main types of cache:

- **Data cache**: A server-side cache for storing data across user requests and deployments. This is persistent but can be revalidated.
- **Request memoization**: A server-side cache for return values of functions if they are called multiple times in a single request.
- **Full route cache**: A server-side cache of Next.js routes. This cache is persistent but can be revalidated.
- **Router cache**: A client-side cache of routes to reduce server requests on navigation, for a single user session or time-based.

The first two types of cache (data cache and request memoization) mainly apply to using the `fetch()` function on the server side to, for example, pull data from a third-party API. However, recently, it is also possible to use these two types of caches for any function by wrapping them with the `unstable_cache()` function. Despite its name, this function can already safely be used in production. It is only called "unstable" because the API might change and require code changes when new Next.js versions are released. See **https://nextjs.org/docs/app/api-reference/functions/unstable_cache** for more information.

NOTE

Alternatively, the React `cache()` *function could be used to memoize return values of functions, but the Next.js* `unstable_cache()` *function is more flexible, allowing us to dynamically revalidate the cache via a path or tag. We are going to learn more about cache revalidation later in this section.*

The full route cache is an additional cache that makes sure that when data doesn't change, we don't even need to re-render pages on the server side so that Next.js can directly return pre-rendered static HTML and the RSC payload. However, invalidating the data cache will also invalidate the corresponding full route cache and trigger a re-render.

The router cache is a client-side cache and is mainly used when the user navigates between pages, allowing us to instantly show pages that they

have already visited without having to fetch them from the server again.

Additionally, if Next.js detects that a page or route only contains static content, it will pre-render and store it as static content. Static content cannot be revalidated anymore, so we need to be careful and ensure that all dynamic content on our apps is considered "dynamic" by Next.js and not accidentally detected as "static" content.

NOTE

In this book, we call this process **static rendering**. *However, on other resources, it may also be called "automatic static optimization" or "static site generation."*

Next.js will opt out of static rendering and consider a page or route dynamic in the following instances:

- When using a dynamic function, such as **`cookies()`**, **`headers()`**, or **`searchParams`**
- When setting **`export const dynamic = 'force-dynamic'`** or **`export const revalidate = 0`**
- When a Route Handler handles a non-GET request

For more in-depth information on the different types of caching, have a look at the Next.js documentation on caching: **https://nextjs.org/docs/app/building-your-application/caching**.

Now, let's explore how static rendering works in practice by looking at how our route behaves in a production build of our app.

Exploring static rendering in API routes

In this chapter, we implemented a Route Handler for getting blog posts. Now, let's explore how this route behaves in dev and production mode:

1. Edit **`src/app/api/v1/posts/route.js`** and add a **`currentTime`** value with **`Date.now()`** to the response, as follows:

```
    return Response.json({ posts, currentTime: Date.now() })
```

2. Refresh the page on **`http://localhost:3000/api/v1/posts`** a couple of times; you will see that **`currentTime`** is always the latest timestamp.
3. Quit the Next.js development server by using *Ctrl* + *C*.
4. Build the Next.js app for production and start it, as follows:

```
$ npm run build
$ npm start
```

5. Refresh the page on **`http://localhost:3000/api/v1/posts`** a couple of times. Now, **`currentTime`** doesn't change at all! Even if we restart the Next.js server, **`currentTime`** still doesn't change. The response of the **`GET /api/v1/posts`** route is statically rendered during build time.

Static rendering works similarly for routes and pages, so pages will also be statically rendered by default. This means that RSCs do *not* require a server, per se; they can also run during build time. We only need a Node.js server if we want to have dynamic pages/routes. This means we could, for example, create a blog or website in Next.js and export a static bundle, allowing us to host it on a simple web server.

NOTE

Exporting a Next.js app as a static bundle can be achieved by specifying the **`output: 'export'`** *option in the* **`next.config.js`** *file.*

Interestingly, if we create a new blog post, our home page *does* get updated. However, that is only the case because **`RootLayout`** uses **`cookies()`** to check if the user is logged in, making all pages on our blog app dynamic (and thus not statically rendered). This can also be seen by looking at the output of **`npm run build`**:

```
● → ~/D/F/ch18 ᚠ main± > npm run build

  > ch16@0.1.0 build
  > next build

     ▲ Next.js 14.1.0
     - Environments: .env

     Creating an optimized production build ...
   ✓ Compiled successfully
   ✓ Linting and checking validity of types
   ✓ Collecting page data
   ✓ Generating static pages (8/8)
   ✓ Collecting build traces
   ✓ Finalizing page optimization

  Route (app)                              Size     First Load JS
  ┌ λ /                                    178 B          91.1 kB
  ├ λ /_not-found                          885 B          85.1 kB
  ├ ○ /api/v1/posts                        0 B                0 B
  ├ λ /create                              143 B          84.3 kB
  ├ λ /login                               875 B            85 kB
  ├ λ /posts/[id]                          144 B          84.3 kB
  └ λ /signup                              875 B            85 kB
  + First Load JS shared by all            84.2 kB
    ├ chunks/69-db30a0e38e2695f2.js        28.9 kB
    ├ chunks/fd9d1056-cc48c28d170fddc2.js  53.4 kB
    └ other shared chunks (total)          1.87 kB


  ○  (Static)   prerendered as static content
  λ  (Dynamic)  server-rendered on demand using Node.js
```

Figure 18.2 – Seeing which routes are statically and dynamically rendered in the build output

As can be seen from *Figure 18.2*, the **`/api/v1/posts`** route is "prerendered as static content," while all other routes are "server-rendered on demand using Node.js."

NOTE

If we wanted to statically render some pages in our blog, we would have to make sure the user bar isn't visible on those pages. For example, we could create a **route group** *(https://nextjs.org/docs/app/building-your-application/routing/route-groups) for all pages that have a user bar, with a separate layout that contains the user bar. Then, we can remove the user bar from the root layout. That way, we could create, for example, an About page that is statically rendered while keeping the rest of the blog dynamic.*

As we have seen, in Next.js, pages and routes are statically rendered by default (if possible). However, in the case of our API route, this is not what we want! We want to be able to dynamically fetch posts from the API. Static rendering and caching in Next.js can be confusing when we're starting out developing apps with it, but it becomes a powerful tool for keeping our apps optimized.

Now, let's learn how to properly handle the cache to make our pages and routes dynamic when they need to be while keeping them cached whenever possible.

Making the route dynamic

To make the route dynamic, we need to set the **`export const dynamic = 'force-dynamic'`** flag on it. Follow these steps:

1. Edit **`src/app/api/v1/posts/route.js`** and add the following code:

```
export const dynamic = 'force-dynamic'
```

2. Quit the currently running Next.js server.
3. Build the Next.js app for production and start it, as follows:

```
$ npm run build
$ npm start
```

4. Refresh the page on **`http://localhost:3000/api/v1/posts`** a couple of times. Now, the API route behaves the same way as it did on the development server!

Unfortunately, we now have completely disabled the cache, so we also don't get any of the benefits of using a cache. Next, we'll learn how to turn on the cache for specific functions.

Caching functions in the data layer

To cache functions from our data layer, we can use the **`unstable_cache()`** function from Next.js. The **`unstable_cache(fetchData, keyParts, options)`** function accepts three arguments:

- **`fetchData`**: The first argument is the function to be called. The function can also have arguments.
- **`keyParts`**: The second argument is an array of unique keys that identify the function in the cache. Arguments that are passed to the function in the first argument will automatically be added to this array as well.
- **`options`**: The third argument is an object containing options for the cache, where we can specify **`tags`** to revalidate the cache later, and a **`revalidate`** timeout to automatically revalidate the cache after a certain number of seconds.

Now, let's enable this cache for all functions where it makes sense. Follow these steps to get started:

1. Edit **`src/data/posts.js`** and import the **`unstable_cache()`** function, aliasing it as **`cache()`**:

```
import { unstable_cache as cache } from 'next/cache'
```

2. Wrap the **`listAllPosts`** function with **`cache()`**, as follows:

```
export const listAllPosts = cache(
  async function listAllPosts() {
    return await Post.find({})
      .sort({ createdAt: 'descending' })
      .populate('author', 'username')
      .lean()
  },
  ['posts', 'listAllPosts'],
  { tags: ['posts'] },
)
```

   As the cache key, we defined an array containing the filename (**`posts`**) and the function name (**`listAllPosts`**) to uniquely identify the function in our data layer. Additionally, we added a **`posts`** tag, which we are going to use later to revalidate the cache when new posts are created.

3. Next, wrap the **`getPostById`** function:

```
export const getPostById = cache(
```

```
  async function getPostById(postId) {
    return await Post.findById(postId).populate('author', 'username').lean()
  },
  ['posts', 'getPostById'],
)
```

4. You may notice that there is now an error when getting posts because **`ObjectId`** from MongoDB is getting serialized into a string by the cache. Edit **`src/components/Post.jsx`** and adjust **`propType`**, as follows:

```
Post.propTypes = {
  _id: PropTypes.string.isRequired,
```

5. Edit **`src/data/users.js`** and import **`unstable_cache`** there:

```
import { unstable_cache as cache } from 'next/cache'
```

6. Wrap the **`getUserInfoById`** function:

```
export const getUserInfoById = cache(
  async function getUserInfoById(userId) {
    const user = await User.findById(userId)
    if (!user) throw new Error('user not found!')
    return { username: user.username }
  },
  ['users', 'getUserInfoById'],
)
```

7. Quit the currently running Next.js server.
8. Rebuild and start the app in production. You will notice that after creating a new post, it does not update the home page (or the API route) anymore:

```
$ npm run build
$ npm start
```

   That's because our posts are now cached!
9. This cache even works in dev mode. Quit the Next.js server and start it again, as follows:

```
$ npm run dev
```

10. Create a new post; you will see that neither the home page nor the API route has the newly created post in the list.

Now that caching has been configured, let's learn how to deal with revalidating the cache (causing data in the cache to be updated).

Revalidating the cache via Server Actions

The best way of dealing with stale data is to revalidate the cache when new data comes in, for example, via Server Actions. To do this, we have two options:

- Revalidating all route segments at a specific path by using the **`revalidatePath`** function
- Revalidating with a specific tag (and thus potentially revalidating multiple paths) by using the **`revalidateTag`** function

Revalidation means that the next time data is requested from the cached function, the function will be called, and new data will be returned and cached (instead of returning previously cached data). Both functions revalidate the data cache and thus revalidate the full router cache and the client-side router cache as well.

Follow these steps to call the **`revalidateTag`** function after creating new posts:

1. Edit **`src/app/create/page.js`** and import the **`revalidateTag`** function:

```
import { revalidateTag } from 'next/cache'
```

2. Inside **`createPostAction`**, call the **`revalidateTag`** function on the **`posts`** tag after creating the new post:

```
  async function createPostAction(formData) {
    'use server'
    const userId = getUserIdByToken(token?.value)
    await initDatabase()
    const post = await createPost(userId, {
      title: formData.get('title'),
      contents: formData.get('contents'),
```

```
    })
    revalidateTag('posts')
    redirect(`/posts/${post._id}`)
  }
```

3. Now, create a new post and go to the home page. You will see that the newly created post appears in the list! The API route will also show the newly created post now.

Revalidating the cache when data is changed via Server Actions is the most direct way of updating the cache. However, sometimes, we will be fetching data from third-party APIs, where revalidating is not possible. We'll explore this case now.

Revalidating the cache via a Webhook

If the data comes from a third-party source, we can revalidate the cache via a Webhook. Webhooks are APIs that can be used as callbacks. For example, when data changes, the third-party source calls our API endpoint to let us know that we need to re-fetch the data.

Integrating a third-party API

Before we can start implementing a Webhook, let's integrate a third-party API into our app. For this example, we are going to use the WorldTimeAPI (**https://worldtimeapi.org/**), but feel free to use any API of your choice.

Let's start implementing a page that fetches from a third-party API:

1. Create a new **`src/app/time/`** folder. Inside it, create a new **`src/app/time/page.js`** file.
2. Edit **`src/app/time/page.js`** and define an asynchronous page component:

```
export default async function TimePage() {
```

3. Inside the component, fetch the current time from the WorldTimeAPI and parse the response as JSON:

```
  const timeRequest = await fetch('https://worldtimeapi.org/api/timezone/UTC'
```

```
  const time = await timeRequest.json()
```

4. Render the current timestamp:

```
  return <div>Current timestamp: {time?.datetime}</div>
}
```

5. If you go to the **`http://localhost:3000/time`** page in your browser, you will see that it shows the current time. However, when refreshing, the time never updates. That is because requests with **`fetch`** are cached by default, similar to what happened after we added **`unstable_cache()`** to our data layer functions.

Implementing the Webhook

Now, let's create a Webhook API endpoint in our app that, when called, revalidates the cache for the third-party data:

1. Create a new **`src/app/api/v1/webhook/`** folder. Inside it, create a new **`src/app/api/v1/webhook/route.js`** file.
2. Edit **`src/app/api/v1/webhook/route.js`** and import the **`revalidatePath`** function:

```
import { revalidatePath } from 'next/cache'
```

3. Now, define a new **`GET`** Route Handler that calls **`revalidatePath`** on the **`/time`** page and then returns a response telling us that it was successful:

```
export async function GET() {
  revalidatePath('/time')
  return Response.json({ ok: true })
}
export const dynamic = 'force-dynamic'
```

Usually, Webhooks are defined as **`POST`** Route Handlers (because they influence the state of the app), but to make it simpler to trigger the Webhook by visiting the page in our browser, we defined it as a **`GET`** Route Handler. A **`POST`** route would opt out of static rendering, but a **`GET`** route does not, so we need to specify **`force-dynamic`**.

4. Visit **`http://localhost:3000/api/v1/webhook`** in your browser, then visit **`http://localhost:3000/time`** again; you should see that the time has been updated! In the real world, we would be adding our Webhook URL to the interface of the third-party website that provides the API.

NOTE

Alternatively, we could add a tag to the request by passing the **`next.tags`** *option in the* **`fetch()`** *function, as follows:* **`fetch('https://worldtimeapi.org/api/timezone/UTC', { next: { tags: ['time'] } })`**. *Then, we could revalidate the cache by calling* **`revalidateTag('time')`**.

As we can see, revalidating the cache using Webhooks works great. However, sometimes, we cannot even add a Webhook to a third-party API. Let's explore what to do when we have no control over the third-party API.

Revalidating the cache periodically

If we have no control whatsoever over the third-party data source, we can tell Next.js to periodically revalidate the cache. Let's set that up now:

1. Edit **`src/app/time/page.js`** and adjust the **`fetch()`** function, adding the **`next.revalidate`** option to it:

```
const timeRequest = await fetch('https://worldtimeapi.org/api/timezone/UTC'
  next: { revalidate: 10 },
})
```

   In this case, we told Next.js to revalidate the data cache the next time the API is requested if at least 10 seconds have passed since the last request.

NOTE

With **`unstable_cache()`**, *we can pass the* **`revalidate`** *option in the third argument. For routes and pages, we can specify* **`export const revalidate = 10`**, *which will revalidate the corresponding route/page.*

2. Refresh the **`http://localhost:3000/time`** page in your browser. You will see the time update. Refresh the page again; the time will not update again. If you refresh after at least 10 seconds, the time will update again.

Now that we have learned about revalidating the cache periodically, let's learn about opting out of caching.

Opting out of caching

Sometimes, you may want to opt out of caching completely for certain requests. To do this, pass the following option to the **`fetch`** function:

```
fetch('<URL>', { cache: 'no-store' })
```

For pages/routes, we can define **`export const dynamic = 'force-dynamic'`** to opt out of the full route cache (the data may still be cached though!).

Now that we've learned how to use the cache in Next.js to optimize our app, let's learn about SEO with Next.js.

SEO with Next.js

In ***Chapter 8***, we learned about SEO in full-stack apps. Next.js provides functionality for SEO out of the box. Let's explore this functionality now, starting with adding dynamic titles and meta tags.

Adding dynamic titles and meta tags

In Next.js, we can statically define metadata by exporting a metadata object from a **`page.js`** file, or we can dynamically define metadata by exporting a **`generateMetadata`** function. We have already added static metadata to the root layout, as can be seen in **`src/app/layout.js`**:

```
export const metadata = {
  title: 'Full-Stack Next.js Blog',
```

```
    description: 'A blog about React and Next.js',
  }
```

Now, let's dynamically generate metadata for our post pages:

1. Edit **`src/app/posts/[id]/page.js`** and define the following function outside of the page component:

```
export async function generateMetadata({ params }) {
  const id = params.id
```

2. Fetch the post; if it does not exist, call **`notFound()`**:

```
  const post = await getPostById(id)
  if (!post) notFound()
```

3. Otherwise, return a title and description:

```
  return {
    title: `${post.title} | Full-Stack Next.js Blog`,
    description: `Written by ${post.author.username}`,
  }
}
```

That's all there is to it! Next.js will set the title and meta tags appropriately for us.

NOTE

Metadata is inherited from layouts. So, it is possible to define defaults for metadata in the layout and then selectively override it for specific pages.

Now that we have successfully added a dynamic title and meta tags, let's continue by creating a **`robots.txt`** file so that search engines know they are allowed to index our blog app.

Creating a robots.txt file

Next.js has two ways of creating a **`robots.txt`** file:

- Creating a static **`robots.txt`** file in **`src/app/robots.txt`**

- Creating a dynamic **`robots.txt`** file by creating a **`src/app/robots.js`** script, which returns a special object that is turned into a **`robots.txt`** file by Next.js

NOTE

If you need a refresher on what a **`robots.txt`** *file is and how search engines work, please check out* ***Chapter 8***.

We are only going to create a static **`robots.txt`** file as there is no need for a dynamic file for now. Follow these steps to get started:

1. Create a new **`src/app/robots.txt`** file.
2. Edit **`src/app/robots.txt`** and add the following contents to allow all crawlers to index all pages:

```
User-agent: *
Allow: /
```

Now that we have created a **`robots.txt`** file, let's create meaningful URLs.

Creating meaningful URLs (slugs)

Now, we are going to create slugs for our blog posts, similar to what we did in ***Chapter 8***. Let's get started:

1. Rename the **`src/app/posts/[id]/`** folder to **`src/app/posts/[...path]/`**. This turns it into a catch-all route, matching everything that comes after **`/posts`**.
2. Edit **`src/app/posts/[...path]/page.js`** and adjust the code to get the first part of the URL (the **`id`** value) from the **`path`** param:

```
export default async function ViewPostPage({ params }) {
  await initDatabase()
  const [id] = params.path
  const post = await getPostById(id)
```

3. Also, adjust the code for the **`generateMetadata`** function:

```
export async function generateMetadata({ params }) {
  const [id] = params.path
```

With that, our router has been set up to accept an optional slug in the URL.

4. Install the **slug** npm package:

```
$ npm install slug@8.2.3
```

5. Edit **src/components/Post.jsx** and import the **slug** function:

```
import slug from 'slug'
```

6. Adjust the link to the blog post by adding the slug, as follows:

```
        <Link href={`/posts/${_id}/${slug(title)}`}>{title}</Link>
```

7. Open a link from the post list; you will see that the URL now contains the slug.

Now that we've made sure our URLs are meaningful, we'll wrap up this section by creating a sitemap for our blog app.

Creating a sitemap

As we learned in ***Chapter 8***, a sitemap contains a list of URLs that are part of an app so that crawlers can easily detect new content and crawl the app more efficiently, making sure that all content on our blog is found.

Follow these steps to set up a dynamic sitemap in Next.js:

1. First, define a **BASE_URL** for our app as an environment variable. Edit **.env** and add the following line:

```
BASE_URL=http://localhost:3000
```

2. Create a new **src/app/sitemap.js** file, where we import the **initDatabase**, **listAllPosts**, and **slug** functions:

```
import { initDatabase } from '@/db/init'
import { listAllPosts } from '@/data/posts'
import slug from 'slug'
```

3. Define and export a new asynchronous function that will generate the sitemap:

```
export default async function sitemap() {
```

4. First, we list all the static pages:

```
  const staticPages = [
    {
      url: `${process.env.BASE_URL}`,
    },
    {
      url: `${process.env.BASE_URL}/create`,
    },
    {
      url: `${process.env.BASE_URL}/login`,
    },
    {
      url: `${process.env.BASE_URL}/signup`,
    },
    {
      url: `${process.env.BASE_URL}/time`,
    },
  ]
```

5. Then, we get all the posts from the database:

```
  await initDatabase()
  const posts = await listAllPosts()
```

6. Generate an entry for each post by building the URL and adding a **`lastModified`** timestamp:

```
  const postsPages = posts.map((post) => ({
    url: `${process.env.BASE_URL}/posts/${post._id}/${slug(post.title)}`,
    lastModified: post.updatedAt,
  }))
```

7. Finally, return **`staticPages`** and **`postsPages`** in an array:

```
  return [...staticPages, ...postsPages]
}
```

8. Go to **`http://localhost:3000/sitemap.xml`** in your browser; you will see that Next.js generated the XML for us from the array of objects!

NOTE

It is best practice to add the sitemap to the **`robots.txt`** *file, but we would need to turn it into a dynamic* **`robots.js`** *file so that we can provide the full URL to the sitemap (using the* **`BASE_URL`** *environment variable). Doing this is left as an exercise for you.*

Now that we've optimized our blog app for search engines, let's learn about optimized image and font loading in Next.js.

Optimized image and font loading in Next.js

Loading images and fonts in an optimized way can be tedious, but Next.js makes it very simple by providing the **`Font`** and **`Image`** components.

The Font component

Often, you'll want to use a specific font for your page to make it unique and stand out. If your font is on Google Fonts, you can have Next.js automatically self-host it for you. No requests will be sent to Google by your browser if you use this feature. Additionally, the fonts will be loaded optimally with zero layout shift.

Let's find out how Google Fonts can be self-hosted with Next.js:

1. We are going to load the **`Inter`** font by importing it from **`next/font/google`**. Edit **`src/app/layout.js`** and add the following import:

```
import { Inter } from 'next/font/google'
```

2. Now, load the font, as follows:

```
const inter = Inter({
  subsets: ['latin'],
  display: 'swap',
})
```

   `Inter` is a variable font, so we don't need to specify the weight that we want to load. If the font isn't a variable font, don't forget to specify the weight. The `display: 'swap'` property means that the font gets an extremely small block period to be loaded. If it does not load by then, a fallback font will be used. Once the font has been loaded, it will be swapped in.

3. Specify the font in the `<html>` tag, as follows:

```
    <html lang='en' className={inter.className}>
```

4. Go to `http://localhost:3000/` in your browser; you will see that our blog app is now using the `Inter` font! See the following screenshot for reference:

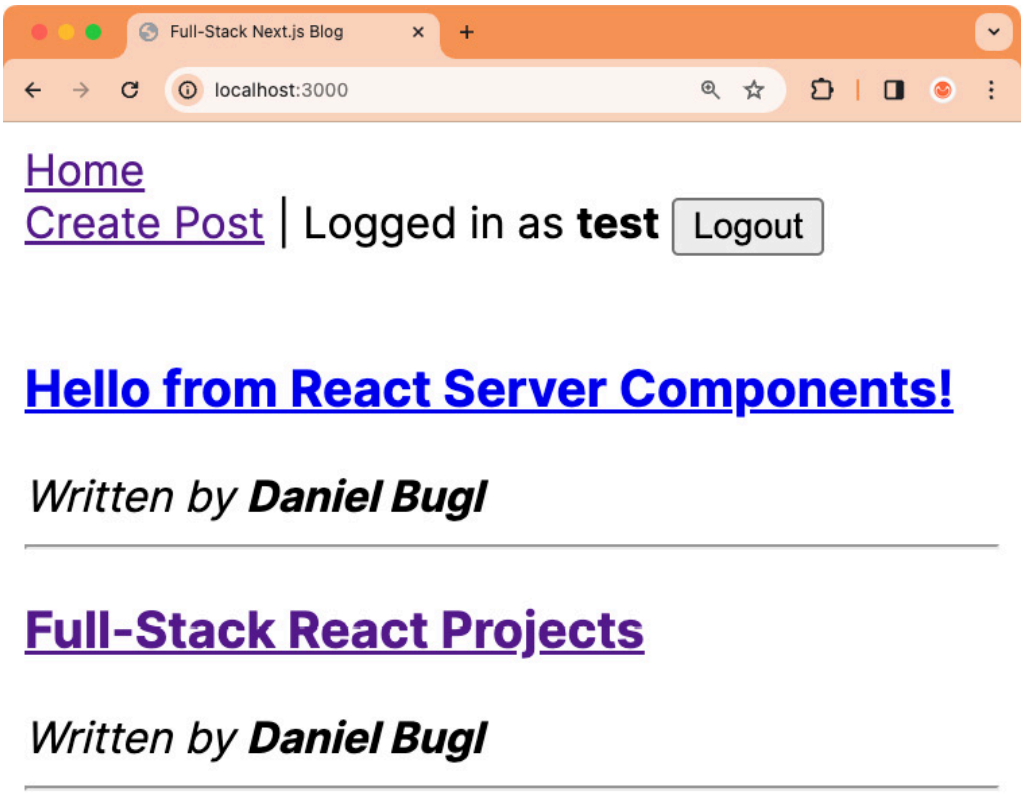

Figure 18.3 – Our blog app rendered with the Inter font

As you can see, it's very simple to use self-hosted Google Fonts with Next.js!

NOTE

If you want to use a font that is not on Google Fonts, use the **`localFont`** *function from* **`next/font/local`**. *This allows you to load a font from a file in your project. For more information on the* **`Font`** *component, check out the Next.js docs:* **https://nextjs.org/docs/app/building-your-application/optimizing/fonts**.

Next, we are going to learn about the **`Image`** component, which allows us to easily load images in an optimized way.

The Image component

Images make up a large portion of the download size of your web application, and can thus have a big impact on the **Last Contentful Paint** (**LCP**) performance. Next.js offers the **`Image`** component, which extends the **`<img>`** element by doing the following:

- Automatically serving resized images for each device and resolution
- Automatically preventing layout shift when images are loading
- Only loading images when they enter the viewport ("lazy loading"), with optional blurred placeholder images
- Offering on-demand resizing for images, even if they are stored remotely

Using the **`Image`** component is simple – just import it and load your images as you would with the **`<img>`** element. Let's try it out now:

1. Get an image to be used as a logo for your blog. Any image can be used, but make sure it is a non-vector format (such as PNG). For vector formats, resizing is not necessary, so you will not see any effect.
2. Save the image as a **`src/app/logo.png`** file.
3. Edit **`src/app/layout.js`** and import the **`Image`** component and the logo:

```
import Image from 'next/image'
```

```
import logo from './logo.png'
```

4. Above the **<nav>** element, render the **<Image>** component, as follows:

```
return (
  <html lang='en' className={inter.className}>
    <body>
      <Image
        src={logo}
        alt='Full-Stack Next.js Blog Logo'
        width={500}
        height={47}
      />
      <nav>
        <Navigation username={user?.username} logoutAction={logoutAction} /
      </nav>
```

   It is important to specify the width and height of the image so that Next.js can infer the correct aspect ratio and prevent layout shift when the image loads in.

5. Go to **http://localhost:3000/** in your browser; you will see the logo being displayed properly! See the following screenshot for reference:

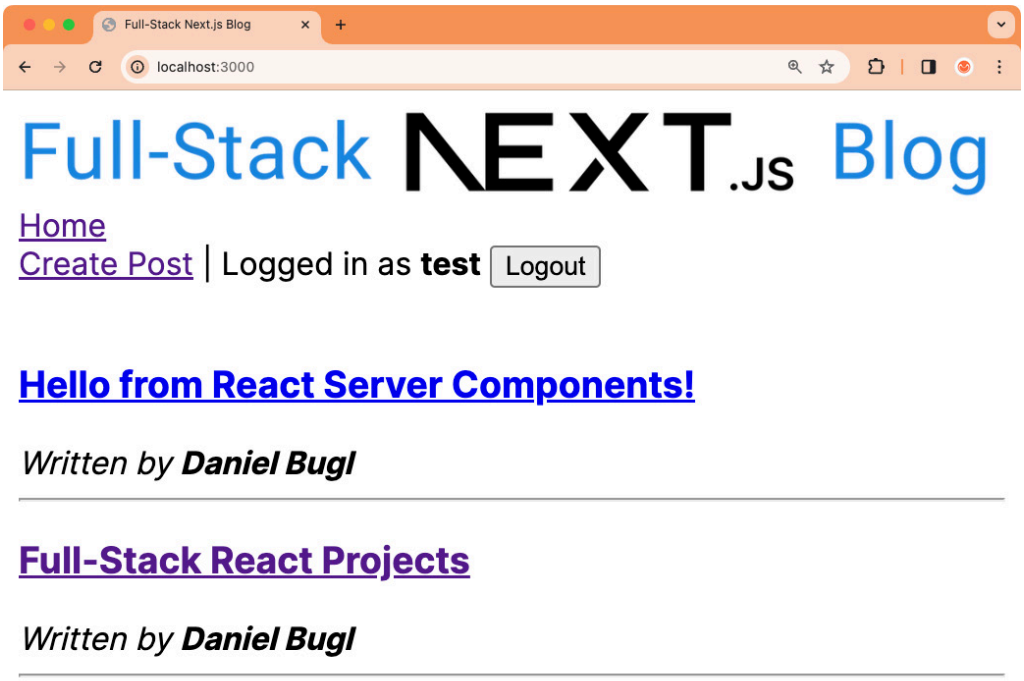

Figure 18.4 – Using the Image component to display a logo for our blog

If you inspect the image in the browser, you will see that it has the **srcset** property with different sizes provided so that the browser can choose

which one to load depending on the screen resolution.

NOTE

In this example, we loaded a local image, but the **`Image`** *component also supports loading images from a remote server, and it will still resize them properly! To use external URLs, allow the remote server by using the* **`images.remotePatterns`** *setting in the* **`next.config.js`** *file, then simply pass a URL instead of a local file to the* **`Image`** *component.*

Summary

In this chapter, we learned how to define API routes in Next.js. Then, we learned about caching, how to revalidate the cache, and how to opt out of the cache. Next, we learned about SEO in Next.js by adding metadata to our pages, creating meaningful URLs, defining a **`robots.txt`** file, and generating a sitemap. Finally, we learned about the **`Font`** and **`Image`** components, which allowed us to load fonts and images easily and optimally in our app.

There are still many more features that Next.js offers that we have not covered yet in this book, such as the following:

- **Internationalization**: Allows us to configure the process of routing and rendering content for multiple languages
- **Middleware**: Allows us to run code before requests are completed, similar to how middleware works in Express
- **Serverless Node.js and Edge runtimes**: Allow us to scale our apps even more by not running a full Node.js server
- **Advanced routing**: Allows us to model complex routing scenarios, such as parallel routes (displaying two pages at once)

In the next chapter, ***Chapter 19***, *Deploying a Next.js App*, we are going to learn how to deploy a Next.js app using Vercel and a custom deployment setup.